

Lecture: Monte Carlo Simulations

Introduction

A computer simulation is a computer model of some (usually) physical process that we are interested in. For example, we might create a simulation of the stock market over time, the population dynamics of a predator and prey species, the processes of star formation, the evolution of HIV resistance in humans, or many other things. In this class, we will use simulations to explore the statistical properties of the statistical methodologies that we study.

Monte Carlo simulations are one of the most useful tools in the statistician's toolkit. They can be used in a wide variety of situations where analytic mathematical solutions would be difficult or impossible. For example, simulations of power, false positive rates, and other characteristics are commonly used by statisticians to evaluate the performance of statistical methods.

Simulations are often very quick and easy in R. An experienced person can often produce a running simulation from scratch in less than 30 minutes. I personally use simulations all of the time in order to explore things that I am interested in.

Steps in Making Simulations

1. Make a set of rules that define how our system of interest works.
2. Implement those rules in a computer program.
3. The output of the computer program will help us understand the implications of the rules that we used to define the system.

The purpose of a simulation is to help us understand the implications of the set of rules that we used to define the system.

If the output of the simulation matches the observed data, then the rules that we used to define the simulation may be a useful approximation of reality.

Example #1: Movement of Nematodes

This is an example from a publication of mine. Nematodes are an animal phylum also sometimes called roundworms. They are typically very small, ranging from roughly 0.1 mm to 2 mm in length. Many species of nematode are parasitic. Species that are parasitic on humans include hookworms, pinworms, and filarias. Other species are parasitic on insects, and thus are used as biological pest control agents. Biological control refers to the use of other organisms for controlling pests. For example, you can buy containers of nematodes to spread in your garden to control certain pest insects:

https://store.doyourownpestcontrol.com/monterey-nematode-control?gclid=CjwKCAjwns_bBRBCEiwA7AVGHjuJI5PTIQ_CJyUWxCyEuy98QfUKC4XVFHa_mCWkhryhkxoC98MQAvD_BwE

A research collaborator of mine at the USDA studies biological control using nematodes. Here is an abstract from a paper:

Abstract

Movement behavior of foraging animals is critical to the determination of their spatial ecology and success in exploiting resources. Individuals sometimes gain advantages by foraging in groups to increase their efficiency in garnering these resources. Group movement behavior has been studied in various vertebrates. In this study we explored the propensity for innate group movement behavior among insect parasitic nematodes. Given that entomopathogenic nematodes benefit from group attack and infection, we hypothesized that the populations would tend to move in aggregate in the absence of extrinsic cues. Movement patterns of entomopathogenic nematodes in sand were investigated when nematodes were applied to a specific locus or when the nematodes emerged naturally from infected insect hosts; six nematode species in two genera were tested (*Heterorhabditis bacteriophora*, *Heterorhabditis indica*, *Steinernema carpocapsae*, *Steinernema feltiae*, *Steinernema glaseri* and *Steinernema riobrave*). Nematodes were applied in aqueous suspension via filter paper discs or in infected insect host cadavers (to mimic emergence in nature). We discovered that nematode dispersal resulted in an aggregated pattern rather than a random or uniform distribution; the only exception was *S. glaseri* when emerging directly from infected hosts (Shapiro-Ilan et al. 2014). The group movement may have been continuous from the point of origin, or it may have been triggered by a propensity to aggregate after a short period of random movement. To our knowledge, this is the first report of group movement behavior in parasitic nematodes in the absence of external stimuli (e.g., without an insect or other apparent biotic or abiotic cue). These findings have implications for nematode spatial distribution and suggest that group behavior is involved in nematode foraging.

Quick summary:

1. The nematodes attack an insect.
2. They burrow into it, kill it, and lay eggs.
3. Eventually, the eggs hatch, the hatchlings burrow out of the insect carcass, and then they move out into the world.
4. After emergence, they look for new prey and the cycle starts again.

My colleague is interested in how they nematodes search for prey, and specifically in the question of whether they disperse from the carcass independently of each other or whether they move as a group. A single nematode is not capable of killing an insect by itself and needs other nematodes to help. Thus, it might be advantageous to move as a group.

In order to study this, we conducted simulations of how nematodes disperse and made comparisons to data. I am just going to tell you about the null hypothesis case of independent movement.

Make the rules (assumptions) that define the simulation

We will assume

1. The nematodes all start at a single point.
2. The move in 2 dimensions.
3. Time is divided into discrete increments. In each increment, the distance that a nematode moves is distributed as Normal(0,1) in the X-dimension and Normal(0,1) in the Y-dimension.
4. Movement is independent between time steps.
5. Movement is independent between nematodes.

When movement follows these rules, it is known as *independent diffusive movement*

Note that these are strong assumptions about how the nematode moves. Do we truly believe that nematodes move this way? Do you think that humans move this way? Humans surely do not move this way. Some other animals sometimes do move in ways that are at least vaguely consistent with these assumptions. For example, flying insects often appear to move about randomly and is not hard to imagine that their movements could follow a normal distribution. In fact it is often observed that animals do move randomly in a way that is similar to 2D diffusive movement, but with heavier tails to the distribution (i.e. more likely to move longer distances).

Implementing the Simulation as an R program

The code below implements this simulation and output is shown. I explain the code in detail below.

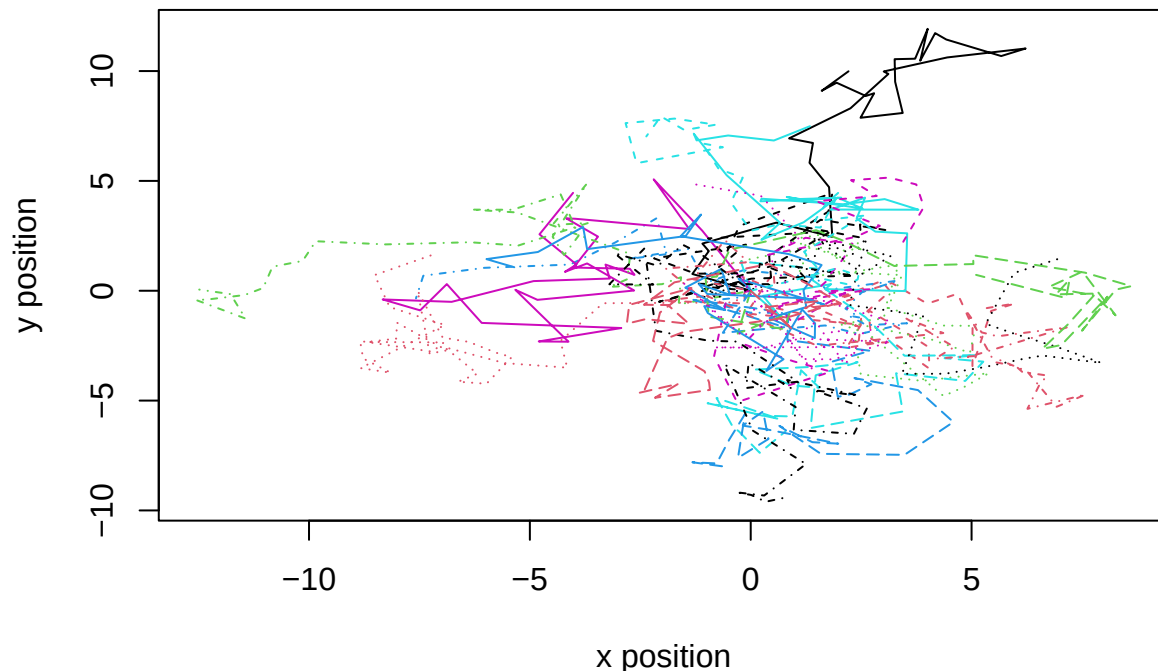
```
T<-30 #number of time steps
n<-20 #number of nematodes

xinc<-matrix(rnorm(T*n),nrow=n) #increments in x position
yinc<-matrix(rnorm(T*n),nrow=n) #increments in y position

xinc[,1]=0 #make all start in the same location
yinc[,1]=0

xdis<-t(apply(xinc,1,cumsum)) #xdis[i,j]=total displacement in x dimension of partical i at time j
ydis<-t(apply(yinc,1,cumsum)) #ydis[i,j]=total displacement in y dimension of partical i at time j

#plot trajectories:
matplot(t(xdis),t(ydis),type="l",xlab="x position",ylab="y position")#xlim=c(-10,10),ylim=c(-10,10))
```



The plot shows how the nematodes would move under the assumptions given. In the research project, we made comparisons of the simulation output to data and formulated other simulations models that used more complicated assumptions about movement.

We will not do this here. Instead, I want to focus on some aspects of the R code.

Generating iid data

Generating iid data is very easy to do in R. In the above example, this is done with the code

```
xinc<-matrix(rnorm(T*n),nrow=n)
```

The code below produces $T \cdot n$ values from the $N(0,1)$ distribution.

```
T=3
n=5
rnorm(T*n)
```

```
## [1] -1.0284343  0.9855746  0.3086078 -0.3201728 -0.6657020  0.5544635
## [7] -0.7128318  1.5133160 -0.6215810  1.2448329 -0.1758057 -0.3887096
## [13] -0.9508154  0.1829622  1.3323387
```

Next, we arrange it into a matrix with rows corresponding to different nematodes:

```
matrix(rnorm(T*n),nrow=n)
```

```
##           [,1]      [,2]      [,3]
## [1,]  1.27114254 0.9412234  1.0441819
## [2,]  0.85949800 1.8256179 -1.0160984
## [3,] -1.28257955 0.6846866 -0.9168179
## [4,]  0.04824706 1.3205475 -1.4802818
## [5,]  0.75512212 0.3613512 -0.5388949
```

We will use this same format many times in producing iid data. Note the compactness of this code. In many other computer languages, we would have to use loops in order to accomplish the same thing. R allows very efficient coding when the same or similar operations are repeated many times.

The iid data in the above example is the movement increments. The x and y movement for each nematode in each time step is independent of all other nematodes and time steps and always follows the same distribution.

We sum the movement increments over time steps to get the location for each nematode at each time step. We do this using the `apply` function.

The apply function

Note the use of the `apply` function:

```
xdis<-t(apply(xinc,1,cumsum)) #xdis[i,j]=total displacement in x dimension of partical i at time j
ydis<-t(apply(yinc,1,cumsum)) #ydis[i,j]=total displacement in y dimension of partical i at time j
```

See the help page for `apply`:

```
?apply
```

The above examples apply the function `cumsum` to the rows (specified by the 1) of `xinc` and `yinc`. This is a very compact and computationally efficient way to apply functions to rows or columns of a matrix.

Note for people coming from other computer languages: Generally, it is best to avoid the use of loops in R and instead use built in functions. R is rather inefficient computationally when it comes to loops, especially nested loops. The built in functions typically use faster computer languages and are optimized for speed.

The function `cumsum` calculates the cumulative sum of a vector:

```
cumsum(1:10)
```

```
## [1]  1  3  6 10 15 21 28 36 45 55
```

The $N(0,1)$ value in each time step is the movement for that time increment. We sum across time steps to get the displacement to a given time point.

```
xinc[1,1:10] #first row, first ten columns of xinc
```

```
## [1] -0.3881282 -0.9250795 0.1014314 0.8821599 0.6912621 -0.1234885
## [7] 2.3276776 -0.4434952 -0.1939896 0.2372622
```

```
cumsum(xinc[1,1:10]) #apply cumsum to first row, first ten columns
```

```
## [1] -0.3881282 -1.3132077 -1.2117762 -0.3296164 0.3616457 0.2381572
## [7] 2.5658348 2.1223396 1.9283500 2.1656122
```

This gives the displacement for the first nematode for the first ten time steps. I just show the first ten time steps to save space.

Next, use the `apply` function to apply the function `cumsum` to each row of `xinc` (that is, to each nematode):

```
apply(xinc[,1:10],1,cumsum) #just showing columns 1:10 so that it will not take up a lot of room in th
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,] -0.3881282 -0.3341424 -2.851203 -1.1608672 0.61084085 -0.8058698
## [2,] -1.3132077 1.0986947 -1.749118 -1.9079773 -1.00219377 0.1638235
## [3,] -1.2117762 2.3064369 -2.284380 -1.7832589 0.90504824 -0.3794054
## [4,] -0.3296164 3.3714293 -2.985090 -2.5379581 -0.02360497 -1.0234082
## [5,] 0.3616457 3.3635283 -4.294300 -2.3593566 0.07910010 -0.6496781
## [6,] 0.2381572 2.6094709 -4.978729 -2.4676758 -0.25307224 -1.3546873
## [7,] 2.5658348 2.2053984 -3.901693 -3.4993718 -0.20682950 -1.5398890
## [8,] 2.1223396 0.9504725 -2.535515 -2.5829839 -1.57852899 -0.5813723
## [9,] 1.9283500 -0.3811380 -3.020836 -1.6374837 -2.14683681 -0.4702002
## [10,] 2.1656122 -2.1570326 -2.405358 0.7098597 -0.71726653 -1.1970380
##           [,7]      [,8]      [,9]      [,10]     [,11]     [,12]
## [1,] -1.21235343 0.02942687 -0.3600379 0.92736537 -0.1019015 -1.4604929
## [2,] -0.02417918 1.52627501 1.2196655 0.36329027 1.0548278 -1.1753411
## [3,] 1.14616238 0.71819739 2.1672424 0.93499278 1.5725429 -0.8116247
## [4,] 1.85342789 -1.35067517 1.8510092 1.08168630 -1.5274761 -1.3276789
## [5,] 1.08648889 -2.47963957 0.7898284 1.30813379 -1.2909408 -1.0524796
## [6,] 2.31910089 -0.10954679 0.7352952 0.45613531 -1.6142784 -1.2024112
## [7,] 2.24522365 0.87435176 0.5566719 0.26772248 -1.8610875 -2.4077218
## [8,] 4.04441428 1.45737457 1.6546808 0.10750673 -1.9851053 -2.0099618
## [9,] 4.23983938 1.54783249 2.3146034 1.37530973 -0.6883815 -1.1236443
## [10,] 4.36212559 0.30865718 3.2993293 -0.07757392 -1.1190253 -0.4954676
##           [,13]     [,14]     [,15]     [,16]     [,17]     [,18]
## [1,] 0.7659668 -0.9208524 2.481751 0.02213368 0.6872623 2.66049360
## [2,] 2.5430139 0.1738643 3.456945 -0.25934490 -0.3417525 1.62819959
## [3,] 3.6955295 -0.8717610 3.175957 -0.09840827 0.1528747 1.83759362
## [4,] 4.4404521 -1.3561880 2.193979 0.77718874 -0.9888944 2.94220675
## [5,] 3.6574288 -0.3477124 1.514874 -0.16860870 -2.0173589 0.05365764
## [6,] 3.6162300 1.4033375 1.004317 -1.07058425 -1.9973124 1.33242502
## [7,] 3.2621881 1.5854477 2.663498 -2.84957876 -2.0650467 2.32416297
## [8,] 2.4514981 0.6678624 2.649339 -2.96866019 -2.5885067 3.26337849
## [9,] 1.6866145 1.8107558 2.911179 -3.45506349 -3.1571831 1.79377102
```

```
## [10,] 0.3078033 1.1737905 2.608881 -4.10341663 -3.4928871 2.74263387
##      [,19]      [,20]
## [1,] -1.1081626 -0.4735682
## [2,] -0.8521735 -2.0353099
## [3,] -1.5214876 -2.2756642
## [4,] -1.2228162 -3.3654818
## [5,] -0.6304497 -4.1295283
## [6,] -0.9383635 -3.4799180
## [7,] -2.3005240 -4.3138196
## [8,] -3.2952852 -3.1068902
## [9,] -3.8014785 -2.5631563
## [10,] -4.3977975 -0.8431608
```

Note that `apply` flips the rows and columns. Now the columns are corresponding to the individual nematodes (note that there are 20 columns) and the rows correspond to time steps.

The function `t` is *matrix transpose*. It flips rows and columns:

```
w=matrix(1:50,nrow=10)
w
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]  1  11  21  31  41
## [2,]  2  12  22  32  42
## [3,]  3  13  23  33  43
## [4,]  4  14  24  34  44
## [5,]  5  15  25  35  45
## [6,]  6  16  26  36  46
## [7,]  7  17  27  37  47
## [8,]  8  18  28  38  48
## [9,]  9  19  29  39  49
## [10,] 10  20  30  40  50
```

```
t(w)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,]  1   2   3   4   5   6   7   8   9  10
## [2,] 11  12  13  14  15  16  17  18  19  20
## [3,] 21  22  23  24  25  26  27  28  29  30
## [4,] 31  32  33  34  35  36  37  38  39  40
## [5,] 41  42  43  44  45  46  47  48  49  50
```

Using the `t` function moves the nematodes back into the row positions:

```
t(apply(xinc[,1:10],1,cumsum))
```

```
##      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,] -0.38812822 -1.31320768 -1.21177625 -0.32961636  0.36164570  0.2381572
## [2,] -0.33414237  1.09869469  2.30643694  3.37142932  3.36352833  2.6094709
## [3,] -2.85120261 -1.74911781 -2.28438039 -2.98508989 -4.29429963 -4.9787288
## [4,] -1.16086715 -1.90797734 -1.78325890 -2.53795810 -2.35935657 -2.4676758
## [5,]  0.61084085 -1.00219377  0.90504824 -0.02360497  0.07910010 -0.2530722
```

```
## [6,] -0.80586982 0.16382346 -0.37940544 -1.02340820 -0.64967808 -1.3546873
## [7,] -1.21235343 -0.02417918 1.14616238 1.85342789 1.08648889 2.3191009
## [8,] 0.02942687 1.52627501 0.71819739 -1.35067517 -2.47963957 -0.1095468
## [9,] -0.36003790 1.21966546 2.16724236 1.85100923 0.78982843 0.7352952
## [10,] 0.92736537 0.36329027 0.93499278 1.08168630 1.30813379 0.4561353
## [11,] -0.10190154 1.05482776 1.57254286 -1.52747610 -1.29094082 -1.6142784
## [12,] -1.46049287 -1.17534107 -0.81162475 -1.32767891 -1.05247958 -1.2024112
## [13,] 0.76596677 2.54301388 3.69552954 4.44045207 3.65742878 3.6162300
## [14,] -0.92085243 0.17386428 -0.87176105 -1.35618799 -0.34771238 1.4033375
## [15,] 2.48175079 3.45694510 3.17595737 2.19397945 1.51487417 1.0043174
## [16,] 0.02213368 -0.25934490 -0.09840827 0.77718874 -0.16860870 -1.0705843
## [17,] 0.68726234 -0.34175249 0.15287472 -0.98889440 -2.01735894 -1.9973124
## [18,] 2.66049360 1.62819959 1.83759362 2.94220675 0.05365764 1.3324250
## [19,] -1.10816257 -0.85217349 -1.52148763 -1.22281619 -0.63044968 -0.9383635
## [20,] -0.47356819 -2.03530988 -2.27566424 -3.36548175 -4.12952833 -3.4799180
##      [,7]      [,8]      [,9]     [,10]
## [1,] 2.5658348 2.1223396 1.9283500 2.16561218
## [2,] 2.2053984 0.9504725 -0.3811380 -2.15703257
## [3,] -3.9016930 -2.5355154 -3.0208363 -2.40535817
## [4,] -3.4993718 -2.5829839 -1.6374837 0.70985971
## [5,] -0.2068295 -1.5785290 -2.1468368 -0.71726653
## [6,] -1.5398890 -0.5813723 -0.4702002 -1.19703804
## [7,] 2.2452236 4.0444143 4.2398394 4.36212559
## [8,] 0.8743518 1.4573746 1.5478325 0.30865718
## [9,] 0.5566719 1.6546808 2.3146034 3.29932930
## [10,] 0.2677225 0.1075067 1.3753097 -0.07757392
## [11,] -1.8610875 -1.9851053 -0.6883815 -1.11902529
## [12,] -2.4077218 -2.0099618 -1.1236443 -0.49546756
## [13,] 3.2621881 2.4514981 1.6866145 0.30780326
## [14,] 1.5854477 0.6678624 1.8107558 1.17379047
## [15,] 2.6634977 2.6493394 2.9111791 2.60888092
## [16,] -2.8495788 -2.9686602 -3.4550635 -4.10341663
## [17,] -2.0650467 -2.5885067 -3.1571831 -3.49288709
## [18,] 2.3241630 3.2633785 1.7937710 2.74263387
## [19,] -2.3005240 -3.2952852 -3.8014785 -4.39779753
## [20,] -4.3138196 -3.1068902 -2.5631563 -0.84316081
```

Putting this all together, we get the x and y displacement relative to the start position for each nematode and time:

```
xdis<-t(apply(xinc,1,cumsum)) #xdis[i,j]=total displacement in x dimension of partical i at time j
ydis<-t(apply(yinc,1,cumsum))

xdis[1:5,1:10] #show for five nematodes and ten time steps
```

```
##      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,] -0.3881282 -1.313208 -1.2117762 -0.32961636 0.3616457 0.2381572
## [2,] -0.3341424 1.098695 2.3064369 3.37142932 3.3635283 2.6094709
## [3,] -2.8512026 -1.749118 -2.2843804 -2.98508989 -4.2942996 -4.9787288
## [4,] -1.1608672 -1.907977 -1.7832589 -2.53795810 -2.3593566 -2.4676758
## [5,] 0.6108409 -1.002194 0.9050482 -0.02360497 0.0791001 -0.2530722
##      [,7]      [,8]      [,9]     [,10]
## [1,] 2.5658348 2.1223396 1.9283500 2.16561218
## [2,] 2.2053984 0.9504725 -0.3811380 -2.15703257
## [3,] -3.9016930 -2.5355154 -3.0208363 -2.40535817
## [4,] -3.4993718 -2.5829839 -1.6374837 0.70985971
## [5,] -0.2068295 -1.5785290 -2.1468368 -0.71726653
```

```
## [1,] 2.5658348 2.1223396 1.928350 2.1656122
## [2,] 2.2053984 0.9504725 -0.381138 -2.1570326
## [3,] -3.9016930 -2.5355154 -3.020836 -2.4053582
## [4,] -3.4993718 -2.5829839 -1.637484 0.7098597
## [5,] -0.2068295 -1.5785290 -2.146837 -0.7172665
```

```
ydis[1:5,1:10]
```

```
##      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,]  0  0.7634539  1.8042429  2.1474366  3.10049907  2.4550941  4.71580448
## [2,]  0 -0.4620298 -0.5798421  0.6283317 -1.61866133 -0.7932937  0.85507030
## [3,]  0  1.2172882  1.5596314  1.8060676  2.63036049  2.3203224  1.46488445
## [4,]  0  0.4157587  0.9333176  0.6030834  0.03683567  0.4630769 -0.06819562
## [5,]  0  1.3283510  0.9991401  0.4889871 -1.20209937 -2.9584746 -2.93365630
##      [,8]      [,9]     [,10]
## [1,]  5.8229928  6.726473  6.936390
## [2,] -1.0343155 -1.547997 -2.599045
## [3,]  2.1753336  1.565748  1.294806
## [4,] -0.8129641 -1.224305 -1.125206
## [5,] -3.0687581 -3.227674 -4.104671
```

Class Exercise

Suppose that we want to modify the simulations for two extensions:

- 1) There is a bias in there movement such that the mean for the x displacement is D rather than 0.
- 2) All nematodes are not identical. Instead, there movement standard deviation comes from a $\text{uniform}(0,2)$ distribution.

Write code to implement these cases (separately). Make plots for case 1) that show how big the bias factor D must get before it makes a noticeable difference in the plot.

Simulation Example #2: Performance of the t-test

We will conduct a simulation study of the performance of the t-test.

Discussion Question: What performance metrics do you think that we should be interested in?

Specify the rules of the simulation

1. We will generate $R=1000$ replicates. In each replicate, we will generate a data set, apply the t-test, and record the result.
2. Assume a two sample t-test, where one sample is generated from $Normal(\mu_1, \sigma_1)$ and the other sample is generated from $Normal(\mu_2, \sigma_2)$
3. In each replicate, We will apply the `t.test` function in R to test for a difference in means between the two sample. We will record the p-value.
4. The estimated probability of rejection is $\frac{\text{number of replicates with p-value} < 0.05}{\text{total number of replicates}}$.
5. If the $\mu_1 = \mu_2$, then the probability of rejection is the type I error rate. If $\mu_1 \neq \mu_2$, then the probability of rejection is the power.

Implementing the Simulation.

The code for this is potentially a bit more complicated because there are two groups. This means that we need to generate iid data for two different groups. I will show several ways to do it.

First approach: replicate function. The `replicate` function allows a very simple way to do simulations. It takes input of the form `replicate(nrep,expr)`, evaluates `expr` a total of `nrep` times, and outputs the results as a vector.

```
nrep=10000 #number of simulation replicates
N1=20 #sample size for sample #1
N2=N1 #sample size for sample #2
mu1=0 #true mean for sample #1
mu2=0 #true mean for sample #2
sd1=1 #true SD for sample #1
sd2=1 #true SD for sample #2

#?replicate

#The replicate function below 1) generates two normal samples,
#2) conducts a t-test, saving the p.value and 3) repeats this nrep times.

x<-replicate(nrep,t.test(rnorm(N1,mu1,sd1),rnorm(N2,mu2,sd2))$p.value)

sum(x<0.05)/nrep #this is the proportion of times that the null hypothesis was rejected.

## [1] 0.0521
```

The expression `t.test(rnorm(N1,mu1,sd1),rnorm(N2,mu2,sd2))$p.value` generates two random samples from a normal distribution, uses a t-test to test for a difference in means, and output the p-value. The `replicate` function repeats this expression `nrep` times and outputs the results in a vector.

The output `sum(x<0.05)/nrep` is the proportion of replicates in which the null hypothesis was rejected. Because the null is true here (i.e. $\mu_1 = \mu_2$), this gives the false positive rate. It is near 0.05 as expected.

Note the use of `sum` with a logical vector:

```
#example:
(y<-runif(10)) #generate 10 values from uniform(0,1)

## [1] 0.9851592 0.2628400 0.6449819 0.3435800 0.4908035 0.0724818 0.7834109
## [8] 0.5314175 0.5743886 0.6834926

y<0.4 #returns T for values less than 0.4 and F for values greater than 0.4

## [1] FALSE TRUE FALSE TRUE FALSE TRUE FALSE FALSE FALSE

sum(y<0.4) #find the number of values less than 0.4.

## [1] 3
```

The expression `y<0.4` returns a logical vector. That is, a vector of T and F according to whether the condition is true.

When we apply the `sum` function to this vector, it converts T to 1 and F to 0. Thus, the `sum` is equal to the number of values where the condition is met. That is, the number of values below 0.4 in the above expression and the number of values less than 0.05 in the simulation code.

Second Approach: Using `apply`. The function `replicate(n,expr)` simply repeats the expression `expr` a total of `n` times and returns the results in a vector or matrix. For example the first example below produces 20 samples of size 10 from a normal distribution. The second example below produces 20 samples of size 10 and takes the mean of each one.

```
replicate(20,rnorm(10))

##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,]  1.7635148 -0.7157806 -1.4688694 -0.1471800  0.1396144 -1.8711150
## [2,]  0.7635658  0.8147108 -1.0963999  0.5815972  0.7923333 -0.3207402
## [3,]  0.5196134  1.0293090 -0.7821812 -1.8585645 -0.9551856  1.3564560
## [4,]  0.6277029  0.3556967 -0.3478187 -0.7553680  0.9009213  1.1840511
## [5,]  1.1252667  2.6514626 -0.3115906  0.9413281 -0.3134603 -0.8408746
## [6,]  0.1606411  0.2108593  0.4203672 -1.5033347  1.5224977 -0.8903240
## [7,]  0.8837536  1.4821866  0.3796108  0.9820460 -0.7441834 -0.4273946
## [8,]  1.1006009 -0.9212769  2.5292841  0.3459144 -2.0200063  1.0935268
## [9,] -0.2589834 -1.5152970  0.9772899  2.2266469 -1.6400260  1.3065391
## [10,] 1.3091144 -0.6075963 -0.2802977 -0.5247581  0.8067685  1.1771570
##           [,7]      [,8]      [,9]     [,10]     [,11]     [,12]
## [1,]  0.1778926  0.81410766  0.92001969  0.4956711  0.01230566  0.94429489
## [2,]  0.2782679  0.48618621  0.28319003  0.8699591  0.81272217  1.56342949
## [3,] -0.2062554 -0.27400461  0.42007781  1.2023720  0.14835635 -0.48435238
## [4,] -2.0055760 -0.82142573  0.68655903  0.5594413 -0.75922074 -1.96729537
## [5,] -1.3394224  1.48666511  1.67833410 -1.1842029  0.70104188  0.05160185
## [6,] -1.2089371 -1.07232898 -1.44473053  2.7072294  0.67598384 -0.88057978
```

```
## [7,] 2.1371827 0.54586487 0.50072883 -0.5499708 -0.21596970 -0.03517068
## [8,] 0.7658050 0.05969046 -0.90626891 -1.1006872 0.37290998 2.15129519
## [9,] -0.1241544 -0.48852440 0.01175533 -0.8397335 -0.78404403 -0.46611960
## [10,] -1.9165715 0.62954644 -0.31249046 0.6377970 1.07178046 -1.93252359
##      [,13]      [,14]      [,15]      [,16]      [,17]      [,18]
## [1,] -1.3200540 0.5617802 -0.3832062 -0.15552187 0.1966215 0.1409846
## [2,] 0.6072100 -0.6157688 -0.5893845 -1.56177449 1.7215228 0.5359339
## [3,] -0.1610112 0.5334942 0.7360040 -1.28801265 0.1302854 -1.9621448
## [4,] -0.2427924 -0.2401469 0.3570173 -0.82577848 -0.4937885 -0.5400768
## [5,] -0.4773743 0.3260757 -0.3700032 0.85138584 1.2298754 -1.2391048
## [6,] -1.3584619 -0.5138295 1.5938099 -0.03431766 -0.3076964 0.1513473
## [7,] -1.1998157 1.3484482 0.5752523 0.48704677 -1.6019964 0.4175103
## [8,] -0.9753463 -0.5398558 -0.1548702 -0.64013981 -1.6532822 0.4556719
## [9,] -0.9353342 0.5029325 0.5324489 -1.01781934 0.2708540 -0.4487079
## [10,] -0.6416174 -0.7680722 0.4677027 0.21459866 -1.1604669 0.1050699
##      [,19]      [,20]
## [1,] -0.5687067 1.05600737
## [2,] 0.4996989 0.04677814
## [3,] -0.7551856 -0.79858963
## [4,] 0.5259926 0.80952369
## [5,] -0.1944621 1.41987849
## [6,] 0.3807411 0.28715079
## [7,] -1.4094744 0.29970504
## [8,] -0.5810195 0.40912456
## [9,] -1.3744517 0.59635390
## [10,] 0.3572455 0.50473426
```

```
replicate(20,mean(rnorm(10)))
```

```
## [1] 0.113545991 0.320708510 0.247801920 -0.148113030 -0.265385143
## [6] -0.175870703 -0.050942332 -0.126017139 0.264659975 0.009874381
## [11] 0.068694701 0.403212169 0.165581405 0.050751087 -0.048351727
## [16] 0.875162623 0.120069760 0.092632443 0.075802500 0.220337682
```

The replicate function is very easy to use if we can conduct the simulation using a single expression, but it won't work for all situations. The next two approaches are more flexible. The first makes use of the `apply` function to conduct the t-test.

```
nrep=1000 #number of simulation replicates
N1=20    #sample size for sample #1
N2=N1    #sample size for sample #2
mu1=0    #true mean for sample #1
mu2=0    #true mean for sample #2
sd1=1    #true SD for sample #1
sd2=1    #true SD for sample #2

#generate data
d1=matrix(rnorm(nrep*N1),nrow=nrep) #d1[i,j]=data point #j in ith replicate for first data set
d2=matrix(rnorm(nrep*N2),nrow=nrep)

#Define a function that conducts a two sample t-test on a vector.
#This takes the first n1 values of x as the first sample and the remaining values as the second sample.
```

```

#It conducts the t-test and returns the p-value.
t_test<-function(x,n1,n2)
{ return(t.test(x[1:n1],x[(n1+1):(n1+n2)]))$p.value) #$p.value extracts the p-value from the output
}

#Combine the two data sets into one matrix and then apply the t_test function
pvalues<-apply(cbind(d1,d2),1,t_test,N1,N2)

#find the proportion of rejected nulls:
sum(pvalues<0.05)/nrep

```

```
## [1] 0.061
```

We needed to define the function `t_test` because I want to use the `apply` function to apply the t-test to the data matrix. There is not a way to apply the built in function `t.test` because the `apply` function requires that I apply the function to the entire row or column and then return a single value. My new function `t_test` is a wrapper function that just repackages the input and output of `t.test` into a form that is more convenient for this application. Creating wrapper functions is often necessary when using `apply`.

Functions `cbind` and `rbind` Note the function `cbind` does “column bind”. That is, it combines the columns to the two matrices together into one matrix:

```
#Examples of cbind
```

```
(m1=matrix(c(1,2,3,4,5,6,7,8,9),nrow=3))
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
(m2=matrix(c(2,4,6,8,10,12,14,16,18),nrow=3))
```

```
##      [,1] [,2] [,3]
## [1,]    2    8   14
## [2,]    4   10   16
## [3,]    6   12   18
```

```
cbind(m1,m2)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1    4    7    2    8   14
## [2,]    2    5    8    4   10   16
## [3,]    3    6    9    6   12   18
```

```
(p0=rnorm(10))
```

```
## [1] -0.5823080  0.2133610 -0.3612283  0.3392773  0.5506830 -0.0983362
## [7] -1.0817481 -0.5293701  0.7396295  0.9422759
```

```
(p1=rnorm(10,mean=1))
```

```
## [1] 0.6646886 -0.8487950 0.7261161 3.0926583 1.1415273 1.1210214
## [7] 1.3856887 1.4405704 0.1328406 1.4112448
```

```
(p2=rnorm(10,mean=2))
```

```
## [1] 2.1061826 3.3938661 1.5397829 1.8055452 1.3161567 -0.9996506
## [7] 1.6565462 1.9838624 1.7869093 2.2766907
```

```
(p3=rnorm(10,mean=3))
```

```
## [1] 3.961113 1.741943 2.802563 2.881007 2.650435 3.568061 2.383932 3.619120
## [9] 1.841597 3.488796
```

```
cbind(p0,p1,p2,p3)
```

```
##           p0           p1           p2           p3
## [1,] -0.5823080 0.6646886 2.1061826 3.961113
## [2,] 0.2133610 -0.8487950 3.3938661 1.741943
## [3,] -0.3612283 0.7261161 1.5397829 2.802563
## [4,] 0.3392773 3.0926583 1.8055452 2.881007
## [5,] 0.5506830 1.1415273 1.3161567 2.650435
## [6,] -0.0983362 1.1210214 -0.9996506 3.568061
## [7,] -1.0817481 1.3856887 1.6565462 2.383932
## [8,] -0.5293701 1.4405704 1.9838624 3.619120
## [9,] 0.7396295 0.1328406 1.7869093 1.841597
## [10,] 0.9422759 1.4112448 2.2766907 3.488796
```

#Note that putting parenthesis around the expressions makes R show the output

The function `rbind` works similarly, but combines by rows instead of columns:

```
(m1=matrix(c(1,2,3,4,5,6,7,8,9),nrow=3))
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
(m2=matrix(c(2,4,6,8,10,12,14,16,18),nrow=3))
```

```
##      [,1] [,2] [,3]
## [1,]    2    8   14
## [2,]    4   10   16
## [3,]    6   12   18
```

```
rbind(m1,m2)
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
## [4,]    2    8   14
## [5,]    4   10   16
## [6,]    6   12   18
```

```
(p0=rnorm(10))
```

```
## [1]  0.05110266  0.48487744  0.13263962  1.37930353 -0.11787271 -0.44955549
## [7] -0.89652786  0.13336544 -0.60815046  2.25094929
```

```
(p1=rnorm(10,mean=1))
```

```
## [1] 0.3350259 2.1533280 0.1929321 0.3802372 0.6441329 0.9784404 3.4437332
## [8] 0.7910281 1.2515133 1.8604295
```

```
(p2=rnorm(10,mean=2))
```

```
## [1] 0.6616162 1.5687387 0.5837544 1.5498331 1.1934468 2.6017230 2.0343319
## [8] 2.9703415 1.9287642 1.4594212
```

```
(p3=rnorm(10,mean=3))
```

```
## [1] 2.103696 2.618815 3.600748 1.618540 3.672905 4.795784 3.273044 2.597139
## [9] 1.944028 4.196378
```

```
rbind(p0,p1,p2,p3)
```

```
##      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## p0 0.05110266 0.4848774 0.1326396 1.3793035 -0.1178727 -0.4495555 -0.8965279
## p1 0.33502587 2.1533280 0.1929321 0.3802372 0.6441329 0.9784404 3.4437332
## p2 0.66161625 1.5687387 0.5837544 1.5498331 1.1934468 2.6017230 2.0343319
## p3 2.10369602 2.6188153 3.6007476 1.6185399 3.6729050 4.7957842 3.2730439
##      [,8]      [,9]     [,10]
## p0 0.1333654 -0.6081505 2.250949
## p1 0.7910281  1.2515133 1.860430
## p2 2.9703415  1.9287642 1.459421
## p3 2.5971385  1.9440278 4.196378
```

Third Approach: Using a for loop The approach above is more flexible than the approach using `replicate`, but still won't work in all situations. This is because using `apply` requires that we can reduce all of our calculations to a single expression that can be applied to the data. We can define our own functions that will make this possible in many complex situations, but it is often more straightforward to use a `for` loop.

The next approach uses a `for` loop to conduct the t-test in each replicate:

```

nrep=1000 #number of simulation replicates
N1=20    #sample size for sample #1
N2=N1    #sample size for sample #2
mu1=0    #true mean for sample #1
mu2=0    #true mean for sample #2
sd1=1    #true SD for sample #1
sd2=1    #true SD for sample #2
pval=vector(length=nrep) #vector to contain p-values for each replicate

#generate data
d1=matrix(rnorm(nrep*N1),nrow=nrep) #d1[i,j]=data point #j in ith replicate for first data set
d2=matrix(rnorm(nrep*N2),nrow=nrep)

for(r in 1:nrep)
{
  pval[r]=t.test(d1[r,],d2[r,])$p.value
}

sum(pval<0.05)/nrep

```

```
## [1] 0.044
```

Here is yet one more way also using a **for** loop. This is how you would do the simulation in many other languages.

```

nrep=1000 #number of simulation replicates
N1=20    #sample size for sample #1
N2=N1    #sample size for sample #2
mu1=0    #true mean for sample #1
mu2=0    #true mean for sample #2
sd1=1    #true SD for sample #1
sd2=1    #true SD for sample #2
pval=vector(length=nrep) #vector to contain p-values for each replicate

for(r in 1:nrep)
{
  d1=rnorm(N1,mu1,sd1) #generate data sets
  d2=rnorm(N2,mu2,sd2)
  pval[r]=t.test(d1,d2)$p.value
}

sum(pval<0.05)/nrep

```

```
## [1] 0.05
```

The difference from the previous example is that we generate the data for each replicate inside the **for** loop, rather than generating it all at once in advance as in the previous example.

The first two approaches are easy to apply when we are working with one dimensional iid data. Then, we can generate all of the data for the simulation in a simple matrix format. For more complex simulations, this won't work. If, for example, the data has multiple variables, then we can't generate data for multiple replicates in a simple matrix format. Or, if the mechanisms for generating the data are more complex (e.g. not from a standard probability distribution), then the **apply** or **replicate** functions will be more difficult to use.

Generally, you should use functions when they are available. These will tend to be faster computationally than your own code and also usually allow more compact code. Thus, the first method using the `replicate` function would be preferred. However, often for more complicated simulations it may be difficult to use built in functions and you may need to use other approaches. Another consideration is ease of understanding of the code. R often allows for very compact code, but it can be challenging to parse out what it is doing when you return to it later. The use of loops is often more straightforward and easy to understand, especially for programmers less familiar with R. Ease of understanding is a very important consideration and should not be dismissed.

Power for the t-test

So far, we have only looked at the type I error rate for the t-test. Next, we will look at power. Our simulation calculates the probability of rejecting the null hypothesis.

Remember: If $\mu_1 = \mu_2$, then the probability of rejecting the null is the probability of a type I error.

If $\mu_1 \neq \mu_2$, then the probability of rejecting the null is the power.

The power will depend on how different μ_1 and μ_2 are from each other relative to the standard deviation. Thus, we will compute plots showing how the power depends on μ_1 and μ_2 .

Note: the effect size for a two sample t-test is

$$\epsilon = \frac{\mu_1 - \mu_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$$

We will keep $\mu_2 = 0$, $\sigma_1 = \sigma_2 = 1$, and $n_1 = n_2$. Thus, the effect size becomes

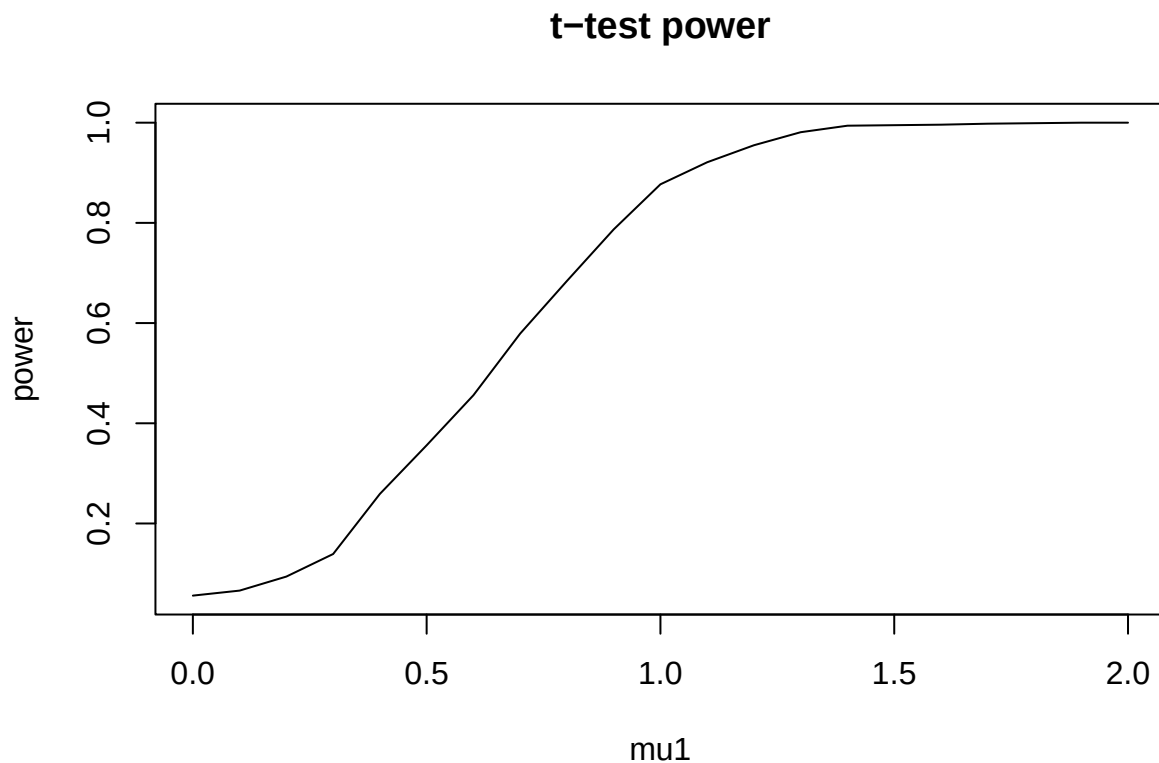
$$\epsilon = \frac{\mu_1 - 0}{\sqrt{\frac{1^2}{n_1} + \frac{1^2}{n_2}}} = \frac{\mu_1}{\sqrt{\frac{2}{n_1}}} = \frac{\mu_1 \sqrt{n_1}}{\sqrt{2}}$$

```
nrep=1000 #number of simulation replicates
N1=20 #sample size for sample #1
N2=N1 #sample size for sample #2
mu1vec=seq(from=0,to=2,by=0.1) #true mean for sample #1
mu2=0 # true mean for sample #2

sd1=1 #true SD for sample #1
sd2=1 #true SD for sample #2
powvec=NULL #will store power for each value of mu1

for(mu1 in mu1vec)
{
  x<-replicate(nrep,t.test(rnorm(N1,mu1,sd1),rnorm(N2,mu2,sd2))$p.value)
  powvec=c(powvec,sum(x<0.05)/nrep) #this is the proportion of times that the null hypothesis was re
}

plot(mu1vec,powvec,type="l",xlab="mu1",ylab="power",main="t-test power")
```

The effect size depends on both sample size and the difference in means. Let's plot both:

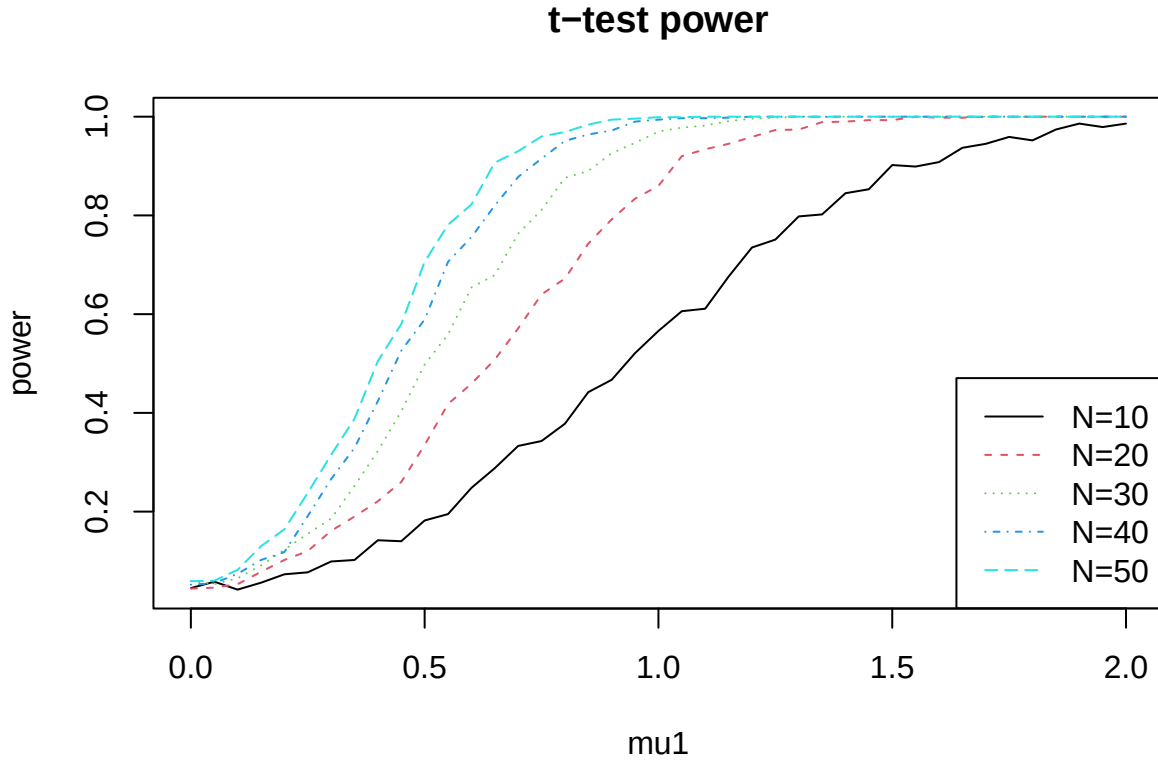
```
nrep=1000 #number of simulation replicates
mu2=0 #true mean for sample #1
mu1vec=seq(from=0,to=2,by=0.05) #true mean for sample #2
sd1=1 #true SD for sample #1
sd2=1 #true SD for sample #2
Nvec=c(10,20,30,40,50)
powmat=NULL #matrix that to store power values. powmat[i,j] =power for ith sample size and jth effect size

for(N1 in Nvec)
{
  N2=N1
  powvec<-NULL

  for(mu1 in mu1vec)
  {
    x<-replicate(nrep,t.test(rnorm(N1,mu1,sd1),rnorm(N2,mu2,sd2))$p.value)
    powvec=c(powvec,sum(x<0.05)/nrep) #this is the proportion of times that the null hypothesis was rejected
  }

  powmat<-cbind(powmat,powvec)
}

matplot(mu1vec,powmat,type="l",xlab="mu1",ylab="power",main="t-test power",lty=1:5,col=1:5)
legend("bottomright",c("N=10","N=20","N=30","N=40","N=50"),lty=1:5,col=1:5)
```



How many replicates should a simulation have?

Suppose that we are conducting a simulation to estimate the probability of some event (e.g. a p-value simulation in which we are estimating the probability of a rejecting the null hypothesis when the null is true). Suppose that p is the probability of the event. If the simulation has R replicates and the event occurs M times, then our estimate $\hat{p}$ for the probability of the event is

$$\hat{p} = \frac{M}{R}$$

This is an estimate because the number M will be variable and we will likely not get exactly the same value every time. We can view the simulation as binomial sampling to estimate the probability p : 1) there is a fixed number R independent replicates; 2) each has two possible outcomes, with the event either occurring or not occurring; 3) the probability of the event is the same in every replicate. Thus, M has a *binomial*(R, p) distribution. From this, we know that

$$E(\hat{p}) = p$$

$$\text{var}(\hat{p}) = \sqrt{\frac{p(1-p)}{R}}$$

If R is large, then a 95% error bound for $\hat{p}$ is

$$\pm 2\sqrt{\frac{p(1-p)}{R}}$$

The table below compiles some values for the error bound as a function of the p and the number of replicates R :

p	R	Error Bound
0.5	100	0.05
0.5	1000	0.016
0.5	10000	0.005
0.05	100	0.022
0.05	1000	0.007
0.05	10000	0.002
1e-06	1000	3e-05
1e-06	10000	1e-05
1e-06	100000	3e-06
1e-06	1e6	1e-06
1e-06	1e7	3e-07

If we want to estimate a probability that is on the order of 0.05, then we need on the order of 1000 replicates. If we want to estimate a small probability such as 1e-6, then we need 10 million or more replicates.

How meaningful are differences in simulated probabilities? Suppose that you are using simulations to compare the statistical power of two different methods. You could use calculations like shown in the table to see whether differences are likely due to chance. For example, if the true power were 50% and you used 1000 replicates, then based on the table above you would expect to get simulated power in roughly the range $50\% \pm 2 * 1.6\% = 50\% \pm 3.2\%$. Thus, if you compare the two methods and their simulated power is within a few percent of each other then the difference is likely sampling variation and not a true difference. If you want to be able to differentiate such power differences then you will need a larger number of replicates.